



東京大学 ブロックチェーン
イノベーション寄付講座

ADVANCED CRYPTOGRAPHY PROGRAM • WEEK 3

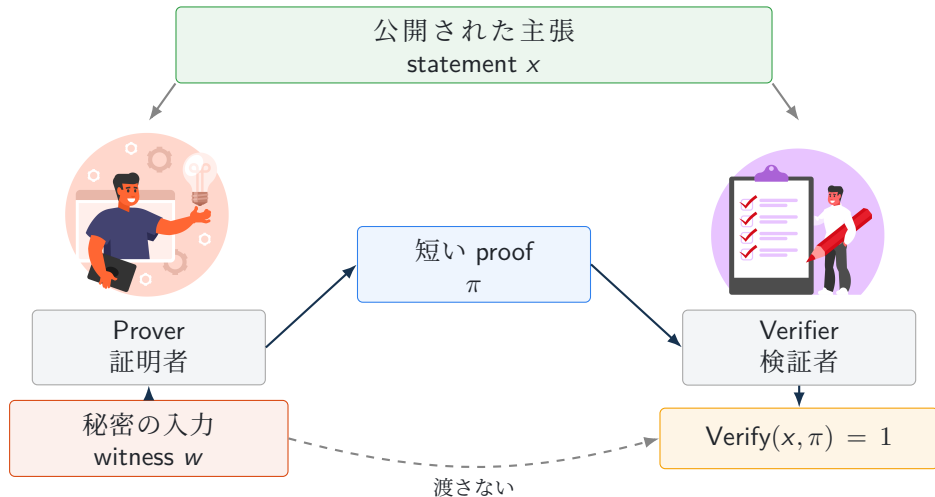
zkSNARK part 1

有限体 • 多項式 • Commitment • Arithmetization

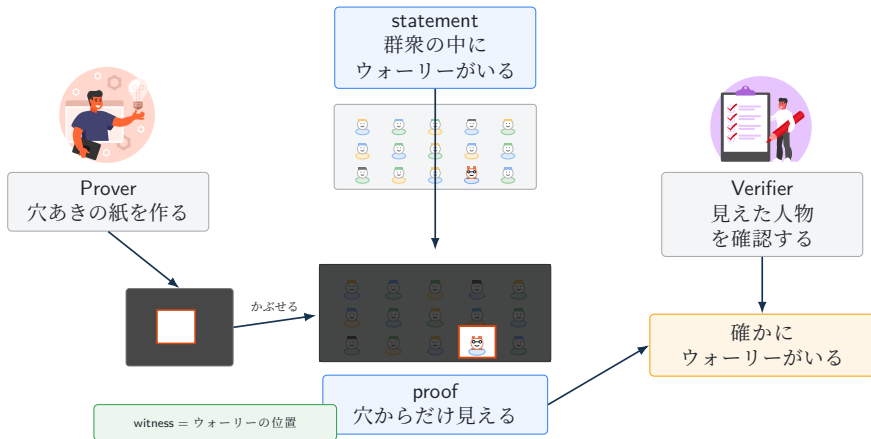
目次

- zkSNARK の導入
- 導入例
- 数の準備
- 多項式の準備
- 楕円曲線/離散対数
- 対話 ZK/FS
- ペアリング
- Commitment
- Arithmetization
- まとめ・補足

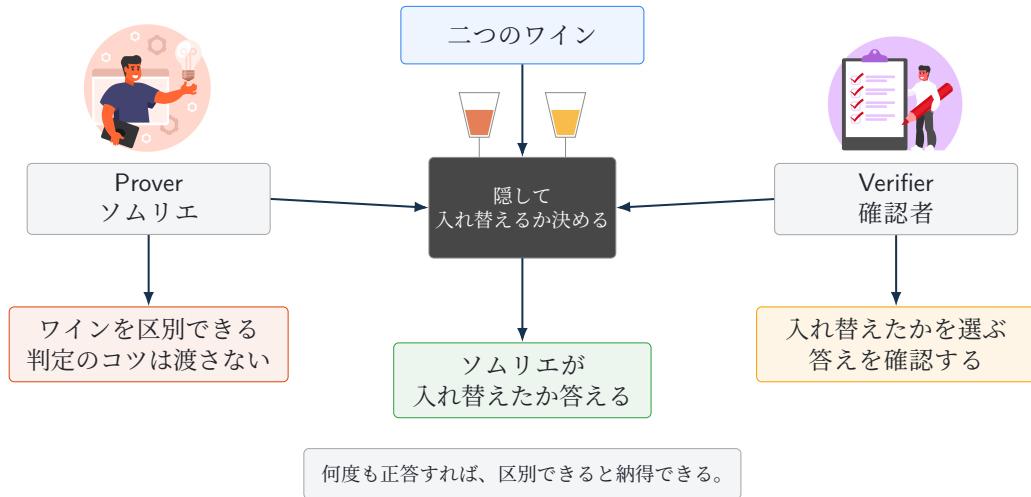
証明者・検証者・秘密・公開入力



ウォーリーを探せの例



ワインソムリエの例



statement と witness

例: 秘密の解を知っている

公開値 $y = 15$ に対して、

$$w^3 + w + 5 = y$$

を満たす秘密の w を知っている、と主張したい。

statement

statement は、検証者にも見える主張である。

$$\exists w : w^3 + w + 5 = 15$$

witness

witness は、主張を成り立たせる秘密情報である。

$$w = 2$$

関係 $R(y, w) = 1$ として書く

証明したい内容は、多くの場合「公開入力 y と witness w が関係 R を満たす」と書ける。

$$R(y, w) = 1$$

R は検査プログラムとして読む。公開値 y と秘密の候補 w を入れるなら、次のように書ける。

```
def R(y, w):  
    # y: public value  
    # w: secret candidate  
    if w**3 + w + 5 == y:  
        return 1  
    return 0
```

たとえば $y = 15$, $w = 2$ なら $2^3 + 2 + 5 = 15$ なので、 $R(15, 2) = 1$ である。

存在主張としての statement

検証者は、秘密そのものを受け取る代わりに存在主張を確認する。

statement: $\exists w$ such that $R(y, w) = 1$

小例

「 $y = 15$ に対して $w^3 + w + 5 = y$ を満たす w を知っている」は、

$$\exists w \in \mathbb{F} : w^3 + w + 5 = 15$$

と書ける。

検証者には「解がある」という事実を渡し、具体的な解 $w = 2$ は証明者の手元に残す。

zkSNARK とは

zkSNARK は、次の性質を同時に満たす証明システムである。

- **Completeness:** 正しい witness を持つ prover の proof は受理される。
- **Knowledge soundness:** 有効な proof を作れるなら、対応する witness を知っているはずである。
- **Zero-knowledge:** proof から witness 由来の余計な情報が漏れない。
- **Succinctness:** proof が短く、verifier は速く検証できる。
- **Non-interactive:** prover と verifier が何度も会話しなくてよい。

簡単に言うと

秘密を隠したまま、条件を満たすことだけを短い proof で示す。

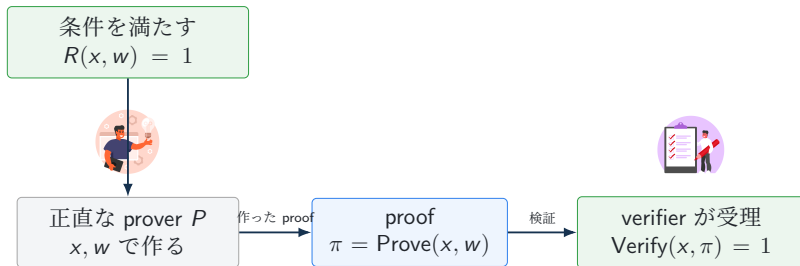
基本用語

statement	公開された主張や条件である。
witness	主張を成り立たせる秘密である。
proof	検証へ渡す短い証拠である。
prover	proof を作る人である。
verifier	proof を検証する人である。

zkSNARK のより正確な定式化 part 1

Completeness は、正しい statement と witness から正直に作った proof を、verifier が受理することを要求する。

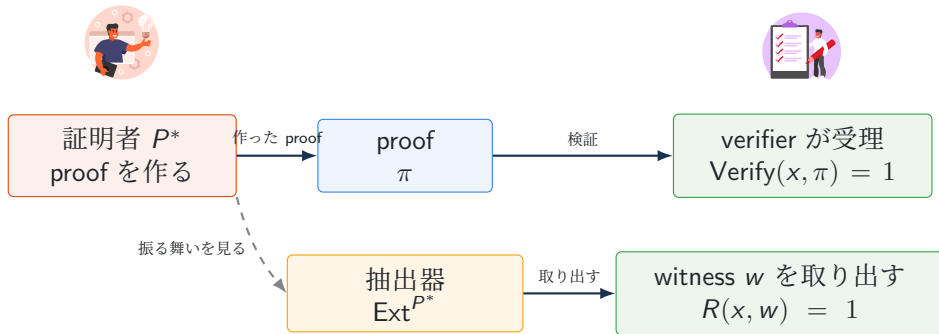
$$R(x, w) = 1 \implies \text{Verify}(x, \text{Prove}(x, w)) = 1$$



ここで x は公開された statement であり、 w は statement を正しくする witness である。この性質は、本当に条件を満たしている prover が、プロトコルのせいで落とされないことを保証する。

zkSNARK のより正確な定式化 part 2

Knowledge soundness は、有効な proof を作れるなら対応する witness を知っているはずだ、という性質である。



証明者 P^* が verifier を十分な確率で受理させられるなら、 P^* の振る舞いにアクセスする抽出器 Ext^{P^*} が $R(x, w) = 1$ となる witness w を取り出せる。

zkSNARK のより正確な定式化 part 3

zero-knowledge は、「本物の実行」と「シミュレーションで作った見かけの実行」を区別できないことを要求する。

$$\text{Real}_{P,V^*}(x, w) \approx \text{Sim}_{S,V^*}(x)$$

$\text{Real}_{P,V^*}(x, w)$ は、prover が witness w を使って作る本物の実行記録を表す。 $\text{Sim}_{S,V^*}(x)$ は、simulator が公開情報 x とシミュレーション用の設定情報から作る見かけの実行記録を表す。zero-knowledge では、verifier はこの二つの記録を見分けられない。

区別できなさの強さには、次の三つの段階がある。

- Perfect は、確率分布が完全に一致していることを意味する。
- Statistical は、確率分布が十分に近いことを意味する。
- Computational は、効率的に区別するアルゴリズムが存在しないことを意味する。

使う道具

zkSNARK では、計算の正しさを短い proof で示すために、次の道具を組み合わせる。

① 有限体

前提として有限体で計算する。値が有限集合の中に閉じていて、丸め誤差がないため、同じ計算を誰が行っても厳密に同じ値になる。

② Commitment

後から内容を書き換えられないように、計算結果や多項式を短い値に固定する。これにより、prover と verifier があらかじめ同意した statement について証明できる。commitment では hash や楕円曲線が使われる。

③ 多項式

多数の制約を一つずつ見る代わりに、多項式の等式としてまとめる。効率的に制約をチェックするために使う。

なぜ数の世界を作り直すのか

verifier が prover の計算を毎回再実行すると、検証の負担が大きくなる。そこで計算を

足し算と掛け算の等式

の集まりへ変換する。

その等式を厳密に扱うために、有限体という閉じた数の世界を使う。

有限体で嬉しいこと

- 丸め誤差がない。
- 計算結果が必ず同じ集合へ戻る。
- 多項式の議論と相性がよい。

$\mathbb{Z}/n\mathbb{Z}$ と $\mathbb{F}_p$

整数を n で割った余りの集合を $\mathbb{Z}/n\mathbb{Z}$ と書く。 $\mathbb{F}_p$ は、素数 p で割った余りを、割り算まで扱える数の世界として見る記号である。

$$\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$$

一般の $\mathbb{Z}/n\mathbb{Z}$

足し算・引き算・掛け算は余りとして計算できる。

ただし、0 以外でいつでも割れるとは限らない。

体として使える条件

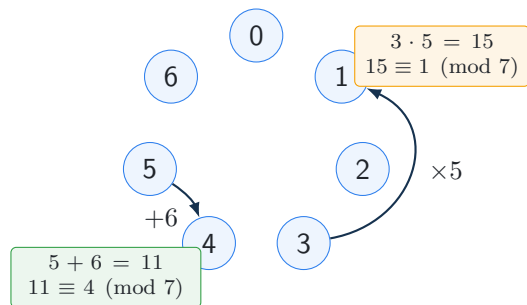
- 足し算・引き算・掛け算で外へ出ない。
- 0 以外の値には逆元がある。
- 割り算を逆元の掛け算として扱える。

有限体は「余りの世界」

$$\mathbb{F}_7 = \{0, 1, 2, 3, 4, 5, 6\}$$

計算結果が7以上なら、7で割った余りへ戻す。

- 足し算・引き算・掛け算が閉じる。
- 0 以外には逆元がある。
- 割り算も、逆元を掛ける操作として扱える。



mod 計算で起きていること

普通の整数では 8 と 1 は違う数。しかし $\mathbb{F}_7$ では、

$$8 \equiv 1 \pmod{7}$$

なので同じ値として扱う。

足し算

$$5 + 6 = 11 \equiv 4 \pmod{7}$$

7 を超えたら余りへ戻る。

掛け算

$$3 \cdot 5 = 15 \equiv 1 \pmod{7}$$

掛け算も同じく余りへ戻る。

素数でない法では割り算が壊れる

$\mathbb{Z}/6\mathbb{Z}$ では、0 以外の二つの値を掛けても 0 になる。

$$2 \cdot 3 \equiv 0 \pmod{6}$$

何が困るか

2 や 3 に逆元がない。逆元がないと、方程式変形や割り算を安定して扱えない。

zk でよく使う選び方

大きな素数 p に対する $\mathbb{F}_p$ を使う。これにより、0 以外の値では割り算を含む代数計算ができる。

逆元と割り算

有限体で割り算をするには、逆元を使う。

$$a \neq 0, \quad a \cdot a^{-1} = 1$$

例: $\mathbb{F}_7$ で $3x = 5$ を解く

$3^{-1} = 5$ なので、両辺に 5 を掛ける。

$$x = 5 \cdot 5 = 25 \equiv 4 \pmod{7}$$

実際に $3 \cdot 4 = 12 \equiv 5 \pmod{7}$ 。

注意

0 には逆元がない。有限体でも 0 割りは定義できない。

$\mathbb{F}_7$ の逆元表

逆元は「掛けると 1 になる値」である。有限体の割り算は、逆元を掛ける操作として読む。

a	1	2	3	4	5	6
a^{-1}	1	4	5	2	3	6

例

$2^{-1} = 4$ である。なぜなら $2 \cdot 4 = 8 \equiv 1 \pmod{7}$ だからである。

拡張 Euclid 互除法で逆元を求める

素数 p と $0 < x < p$ は互いに素なので、

$$ax + bp = 1$$

を満たす整数 a, b が見つかる。
 $\text{mod } p$ で見ると $bp \equiv 0$ なので、

$$ax \equiv 1 \pmod{p}$$

となり、 a が x の逆元になる。

例: $\mathbb{F}_{13}$ で 2^{-1}

$13 = 6 \cdot 2 + 1$ より $1 = 13 - 6 \cdot 2$ 。したがって $-6 \equiv 7 \pmod{13}$ なので、 $2^{-1} = 7$ 。

なぜ有限体を使うのか

- zkSNARK では、計算を数式として厳密に扱う必要がある。
- 実数や浮動小数点数は、丸め誤差や連続性の扱いが難しい。
- 有限体では、すべての計算が厳密で、決まった集合の中に戻る。
- そのため「計算が正しい」を代数的な等式として扱いやすい。

プログラムの条件 → 有限体上の等式

巡回群と位数

$\mathbb{F}_p^* = \mathbb{F}_p \setminus \{0\}$ は掛け算について群になる。ある元 g を何度も掛けることで全体を巡れるとき、 g を生成元と呼ぶ。

$$\langle g \rangle = \{g^0, g^1, g^2, \dots\}$$

- 群の位数は、その群に含まれる要素の数である。たとえば $|\mathbb{F}_p^*| = p - 1$ 。
- 元 g の位数は、 $g^n = 1$ となる最小の正整数 n である。
- g の位数が $p - 1$ なら、 $g^0, g^1, \dots, g^{p-2}$ で $\mathbb{F}_p^*$ 全体を生成する。

群とは、次の性質を持つ集合と演算の組である。

- 演算しても集合の外へ出ない。
- 単位元がある。
- 各要素に逆元がある。
- 括弧の付け方を変えても結果が変わらない。

なぜ多項式が出てくるのか

やりたいこと

たくさんの値や等式を、短い検査へ圧縮したい。

そのまま見る

- すべての行を確認する。
- データが大きいと検証の負担が増える。
- verifier が計算を再実行する方向へ戻ってしまう。

多項式で見る

- 値の列を多項式として表す。
- 多項式の性質でまとめて検査する。
- ランダム点で嘘を見抜きやすい。

多項式は値の列を扱う言語

多項式

$$f(X) = 3X^2 + 2X + 5$$

は二つの見方を持つ。

係数表現

$(5, 2, 3)$

定数項から順に持つ。

評価表現

$(f(x_0), f(x_1), \dots)$

点での値の列として持つ。

重要事実

次数 d 以下の多項式は、異なる $d + 1$ 点での値が決まれば一意に決まる。

補間: 点から式を復元する

1 次式 $f(X) = aX + b$ は 2 点で決まる。

$$f(1) = 3, \quad f(2) = 5$$

なら、傾きは 2、切片は 1 なので、

$$f(X) = 2X + 1.$$

次数が低い多項式ほど、少ない点で形が決まる。後で、値の列を多項式として扱うときの入口になる。

Lagrange 補間の式

異なる点 $x_0, \dots, x_d$ と値 $y_0, \dots, y_d$ があるとする。まず、Lagrange 基底 L_i は評価点で次の振る舞いをするように作る。

$$L_i(x_j) = \begin{cases} 1 & (i = j), \\ 0 & (i \neq j). \end{cases}$$

この性質があれば、 $f(x_i) = y_i$ を満たす多項式を

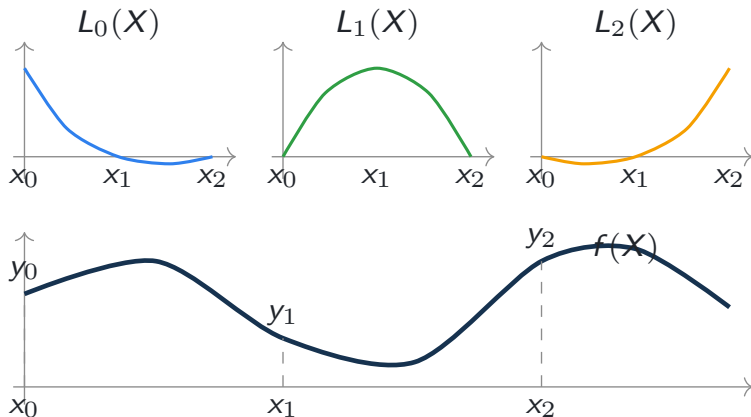
$$f(X) = \sum_{i=0}^d y_i L_i(X)$$

と書ける。
ちなみに、各 $L_i(X)$ は具体的には

$$L_i(X) = \prod_{\substack{0 \leq j \leq d \\ j \neq i}} \frac{X - x_j}{x_i - x_j}$$

と書ける。有限体では $x_i - x_j \neq 0$ なら逆元で割れる。

Lagrange 基底で $f(X)$ を作る



$$f(X) = y_0 L_0(X) + y_1 L_1(X) + y_2 L_2(X)$$

2 次多項式は 3 点で決まる

2 次以下の多項式

$$f(X) = aX^2 + bX + c$$

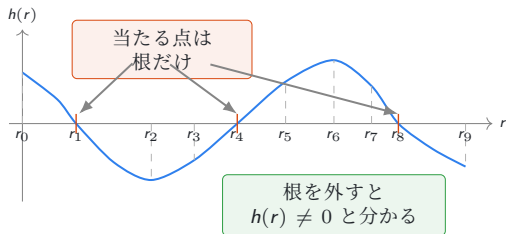
には未知数が 3 つある。

異なる 3 点での値が分かれば、 a, b, c を一意に決められる。

$$f(x_0) = y_0, \quad f(x_1) = y_1, \quad f(x_2) = y_2$$

2 点だけでは候補が残るが、3 点あると 2 次以下の多項式は一つに定まる。

Schwartz-Zippel の補題

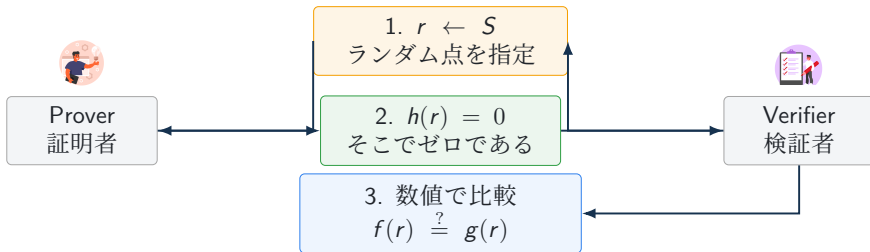


二つの多項式 f, g が等しいかを調べるために、差 $h(X) = f(X) - g(X)$ を見る。

補題の形

$$h \neq 0 \implies \Pr_{r \leftarrow S}[h(r) = 0] \leq \frac{\deg(h)}{|S|}$$

非ゼロ多項式の根は高々次数個なので、集合 S が大きいほどランダム点が根に当たりにくい。



フーリエ変換とは

フーリエ変換は、同じ対象を二つの表し方で行き来する変換として読む。多項式では、係数標識と評価標識のどちらで見てもよい。

表し方	何を持つか	読み方
係数標識	$a = (a_0, \dots, a_{n-1})$	$f(X) = \sum_{i=0}^{n-1} a_i X^i$ として見る。
評価標識	$y = (f(1), f(\omega), \dots, f(\omega^{n-1}))$	決まった点での値の列として見る。

有限体 $\mathbb{F}_p$ の中で、ある元 ω が

$$\omega^n = 1, \quad \omega^0, \omega^1, \dots, \omega^{n-1} \text{ がすべて異なる}$$

を満たすとき、 ω を位数 n の 1 の根と呼ぶ。

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

を規則的な評価点の集合として使う。係数標識から評価標識へ移る操作が、ここで扱う有限体上のフーリエ変換である。

参考: 複素数のフーリエ変換

物理や信号処理では、連続変数の関数を複素指数関数で分解するフーリエ変換がよく使われる。典型的には次のような積分で書く。

$$\widehat{f}(\xi) = \int_{-\infty}^{\infty} f(t) e^{-2\pi i \xi t} dt$$

離散的な場合には、 $\zeta = e^{2\pi i/n}$ を使って

$$\widehat{a}_j = \sum_{i=0}^{n-1} a_i \zeta^{ij}$$

のような和で書ける。

今回使う有限体上のフーリエ変換は、この式の ζ を有限体の位数 n の 1 の根 ω に置き換えたものとして読む。角度や波ではなく、掛け算で一周する点列 $1, \omega, \dots, \omega^{n-1}$ を使う。

有限体上のフーリエ変換を定義する

係数列 $a = (a_0, a_1, \dots, a_{n-1})$ は、次数 n 未満の多項式

$$f(X) = \sum_{i=0}^{n-1} a_i X^i$$

を表す。位数 n の 1 の根 ω に対して、有限体上のフーリエ変換と逆変換を次で定義する。

$$\text{フーリエ変換:} \quad y_j = f(\omega^j) = \sum_{i=0}^{n-1} a_i \omega^{ij}, \quad j = 0, \dots, n-1.$$

$$\text{逆変換:} \quad a_i = \frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij}, \quad i = 0, \dots, n-1.$$

ここで $1/n$ は、有限体の中での n の逆元である。つまり、フーリエ変換は係数列 a から評価値列 y を作り、逆変換は評価値列 y から係数列 a を戻す。

逆変換で使う補題

n は $\mathbb{F}_p$ で逆元を持つとする。位数 n の 1 の根 ω について、次が成り立つ。

$$S_k = \sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & (k \equiv 0 \pmod{n}), \\ 0 & (k \not\equiv 0 \pmod{n}). \end{cases}$$

$$\begin{aligned} k \not\equiv 0 \pmod{n}: \quad (1 - \omega^k) S_k &= \sum_{j=0}^{n-1} \omega^{kj} - \sum_{j=0}^{n-1} \omega^{k(j+1)} \\ &= 1 - \omega^{kn} = 0. \end{aligned}$$

$\omega^k \neq 1$ なので $S_k = 0$ である。 $k \equiv 0$ のときは各項が 1 なので $S_k = n$ である。

逆変換が元の係数を返すこと

$y_j = f(\omega^j) = \sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j}$ とすると、

$$\begin{aligned}\frac{1}{n} \sum_{j=0}^{n-1} y_j \omega^{-ij} &= \frac{1}{n} \sum_{j=0}^{n-1} \left(\sum_{\ell=0}^{n-1} a_\ell \omega^{\ell j} \right) \omega^{-ij} \\ &= \frac{1}{n} \sum_{\ell=0}^{n-1} a_\ell \sum_{j=0}^{n-1} \omega^{(\ell-i)j} \\ &= \frac{1}{n} \left(a_i n + \sum_{\ell \neq i} a_\ell \cdot 0 \right) = a_i.\end{aligned}$$

小例: $\mathbb{F}_7$ で 3 点に評価する

$\mathbb{F}_7$ では $\omega = 2$ が位数 3 の 1 の根になる。

$$2^0 = 1, \quad 2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 1$$

多項式

$$f(X) = 3 + X + 2X^2$$

を $H = \{1, 2, 4\}$ 上で評価すると、

$$f(1) = 6, \quad f(2) = 6, \quad f(4) = 4 \pmod{7}$$

何を変換したか

係数列 $(3, 1, 2)$ を、評価値列 $(6, 6, 4)$ へ変換した。

FFT が必要になる場面

証明生成では多項式を何度も行き来する

制約を作るときは値の列として見たい。多項式の性質を使うときは係数や評価点で見たい。

値の列 $\iff$ 多項式 $\iff$ 別の点での評価

ナイーブにやると

点ごとに一から計算するので、大きな回路では負担が増える。

FFT を使うと

評価点の規則性を利用して、まとめて速く変換できる。

FFT/IFFTは何を速くするのか

変換したいもの

係数表現 $\iff$ 評価表現

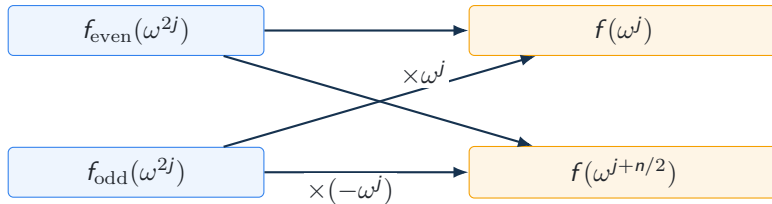
方法	計算量の直感	使う場面
ナイーブ評価 FFT/IFFT	各点で一から計算するので $O(n^2)$ 点集合の対称性で分割統治し $O(n \log n)$	小さい教材例、テスト 大規模な証明生成、多点評価、補間

$$f(\omega^j) = \sum_{i=0}^{n-1} a_i \omega^{ij}$$

FFT の蝶形

FFT では多項式を偶数次と奇数次に分ける。

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2)$$



必要な条件

この蝶形では n が偶数で、 ω が位数 n の 1 の根であり、 $\omega^{n/2} = -1$ となることを使う。
radix-2 FFT では典型的に $n = 2^m$ の評価領域を選ぶ。

群: 足し算だけできる世界

群は「足し算のような操作」が定義された集合である。楕円曲線暗号では、曲線上の点を足す。

点の足し算

$$P + Q = R$$

点と点を足すと、また点になる。

スカラー倍

$$xP = P + \dots + P$$

P を x 回足す操作として読む。

楕円曲線を式で見る

暗号でよく使う楕円曲線は、有限体 $\mathbb{F}_p$ 上の点集合として定義される。

$$y^2 = x^3 + ax + b$$

滑らかさの条件

$$4a^3 + 27b^2 \neq 0$$

この条件により、特異点のない曲線として扱える。

$$E(k) = \{(x, y) \in k^2 \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

$\mathcal{O}$ は無限遠点であり、足し算の単位元として働く。

点の足し算の式: 3つ目の交点

図は実数上での直感である。暗号では同じ式を有限体で $\text{mod } p$ 計算する。

$P = (x_1, y_1)$ 、 $Q = (x_2, y_2)$ とする。 $P \neq Q$ かつ $x_1 \neq x_2$ の通常ケースでは、

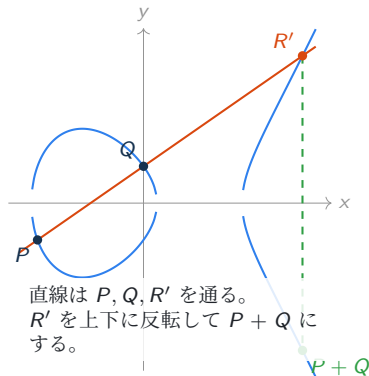
$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}$$

とおく。2点を通る直線を $y = \lambda(x - x_1) + y_1$ と書き、これを楕円曲線 $y^2 = x^3 + ax + b$ に代入すると、

$$\{\lambda(x - x_1) + y_1\}^2 = x^3 + ax + b$$

となり、 x についての3次方程式になる。

3つの解は、直線と曲線の交点の x 座標なので、 x_1, x_2, x_3 である。



点の足し算の式: 反転して和を作る

代入して得た 3 次方程式を整理すると、 x^2 の係数は $-\lambda^2$ になる。根と係数の関係から

$$x_1 + x_2 + x_3 = \lambda^2$$

なので、

$$x_3 = \lambda^2 - x_1 - x_2$$

である。

3 つ目の交点を $R' = (x_3, y')$ とおく。この点は直線上にあるので、

$$y' = \lambda(x_3 - x_1) + y_1$$

である。楕円曲線の足し算では、この点を上下反転した点を $P + Q = (x_3, y_3)$ とする。したがって

$$y_3 = -y' = -\{\lambda(x_3 - x_1) + y_1\} = \lambda(x_1 - x_3) - y_1.$$

$P = Q$ の点倍算では、直線の代わりに接線を使い、

$$\lambda = \frac{3x_1^2 + a}{2y_1}$$

とする。ただし $2y_1 \neq 0$ の場合である。 $y_1 = 0$ なら接線は垂直になり、 $2P = \mathcal{O}$ と扱う。割り算は有限体の逆元で行う。

楕円曲線のスカラー倍

xP は P を x 回足す操作である。実装では、整数の二進展開を使って倍々に作る。

例

$$13P = 8P + 4P + P$$

まず $2P, 4P, 8P$ を doubling で作り、必要なものだけ足す。

この方法なら、計算回数はおおよそ $\log x$ に比例する。

楕円曲線の離散対数問題

楕円曲線暗号では、曲線上の点を足し算できる。

- 点 P と Q を足して $P + Q$ を得る。
- 点 P を x 回足して xP を得る。

Given P and $Q = xP$, find x

この問題を楕円曲線上の離散対数問題と呼ぶ。

暗号では、基点 P が生成する大きな素數位数 q の部分群を使い、 $x \in \mathbb{Z}_q$ として扱う。

一方向性がある

$x \mapsto Q = xP$ は速い

$P, Q = xP \nrightarrow x$ は難しい

これは hash と同じ見方である。hash でも $m \mapsto \text{Hash}(m)$ は速いが、 $\text{Hash}(m)$ から m を戻すのは難しい。

楕円曲線では、入力をスカラー x 、出力を点 xP としてこの構造を使う。

離散対数問題・Diffie–Hellman 系の問題

楕円曲線では点の足し算を使うため、乗法記法の g^a に対応するものを加法記法の aP と書く。
Alice が aP 、Bob が bP を公開し、共通の点 abP を得たい場面を考える。

問題	乗法記法	加法記法
離散対数問題	g, g^a から a を求める	P, aP から a を求める
計算 Diffie–Hellman 問題	g, g^a, g^b から g^{ab} を求める	P, aP, bP から abP を求める
判定 Diffie–Hellman 問題	g, g^a, g^b, T から $T = g^{ab}$ かランダムな g^c かを判定する	P, aP, bP, T から $T = abP$ かランダムな cP かを判定する

記法は違っても、秘密スカラーを公開できる群要素へ写すという構造は同じである。

Schnorr protocol: part 1, 問題設定

公開情報として、巡回群の生成元 g と

$$y = g^x$$

がある。prover は秘密の x を知っていることを示したい。

目的

x を明かさずに、 y が g^x という形で作られていることを納得させる。

ここでは乗法記法で書く。楕円曲線では g^x を xP 、 g^r を rP と読めばよい。群の位数を q とし、指数 r, x, b は $\text{mod } q$ で計算する。

Schnorr protocol: part 2, 対話式プロトコル

- ① prover は乱数 r を選び、 $a := g^r$ を verifier へ送る。
- ② verifier はチャレンジ b をランダムに選んで送る。
- ③ prover は $c := r + bx$ を返す。
- ④ verifier は

$$g^c = ay^b$$

を確認する。

完全性

本当に $y = g^x$ なら、

$$g^c = g^{r+bx} = g^r (g^x)^b = ay^b$$

となる。

Schnorr protocol: part 3, 知識健全性

同じ a に対して、 $b = 0$ と $b = 1$ の両方に答えられるなら、秘密 x を取り出せる。ここではまず 2 値チャレンジの簡単な場合を見る。

$$c_0 = r, \quad c_1 = r + x$$

したがって

$$x = c_1 - c_0$$

である。

一般には、同じ a に対する異なるチャレンジ $b \neq b'$ と応答 c, c' から

$$x = (c - c')(b - b')^{-1} \pmod{q}$$

を取り出せる。

巻き戻しの見方

prover を同じ a を送った時点まで戻し、別のチャレンジを投げる。両方に答えられるなら、prover は実質的に x を知っている。

Schnorr protocol: part 4, honest-verifier zero-knowledge 性

正直な verifier がランダムな b を送る場合、verifier が見る会話 (a, b, c) は秘密 x を使わずに作れる形をしている。

simulator の作り方

b, c を先にランダムに選び、

$$a := g^c y^{-b}$$

と置く。

本物の実行では $c = r + bx$ なので、

$$a = g^r = g^{c-bx} = g^c (g^x)^{-b} = g^c y^{-b}$$

と同じ形になる。

すると検証式

$$g^c = ay^b$$

は必ず成り立つ。公開情報だけで本物と見分けにくい会話を作れることが、honest-verifier zero-knowledge の核になる。

Schnorr protocol: part 5, Fiat-Shamir 変換

対話式プロトコルでは、verifier が後からランダムなチャレンジ b を送る。Fiat-Shamir 変換では、このチャレンジをハッシュで作る。

$$b := H(\text{ctx}, g, y, a)$$

ctx に入れるもの

プロトコル名、曲線や群の種類、証明したい statement などの文脈情報を入れる。

transcript は公開された会話ログである。transcript hash により、verifier が送るはずのチャレンジを公開情報から決定論的に再計算できる。

Schnorr protocol: part 6, 非対話 proof への変換

Fiat-Shamir 後の prover:

- ① 乱数 r を選び、 $a := g^r$ を計算する。
- ② $b := H(\text{ctx}, g, y, a)$ を計算する。
- ③ $c := r + bx \pmod{q}$ を計算する。
- ④ proof として $\pi = (a, c)$ を送る。

verifier は a から同じ b を再計算し、

$$g^c = ay^b$$

を確認する。

実装で重要なこと

domain separation、値の一意なエンコード、十分に大きなチャレンジ空間を用意する。

ペアリング

ペアリングは、二つの群の要素を別の群へ写す演算である。

$$e : G_1 \times G_2 \rightarrow G_T$$

性質は次のように整理できる。

- 双線形性:

$$e(aP, Q) = e(P, Q)^a, \quad e(P, bQ) = e(P, Q)^b, \quad e(aP, bQ) = e(P, Q)^{ab}.$$

- 非退化性: 生成元 P, Q に対して $e(P, Q)$ が非自明な値になる。
- 効率的計算可能性: P, Q から $e(P, Q)$ を効率よく計算できる。
- 実用では、 G_1, G_2, G_T を別の群として扱う非対称ペアリングがよく使われる。

P, Q, aP, bQ から秘密スカラーの積 ab を直接取り出すわけではない。ペアリングで得られる G_T の要素 $e(P, Q)^{ab}$ を使い、同じ積に対応するかを等式として比較できる。

Commitment は zkSNARK で何をしているか

証明システムでは、大きな対象が何度も出る。

- 制約の列
- 実行トレース
- 多項式
- wire 値や selector

役割

commitment は、大きな対象を短い値で代表させ、必要な点だけを open するために使う。



開示する部分だけを
検証対象にする

Commitment scheme

commitment は、値 m を短い値 c に固定し、後で m と乱数 r を見せて確認できる仕組みである。

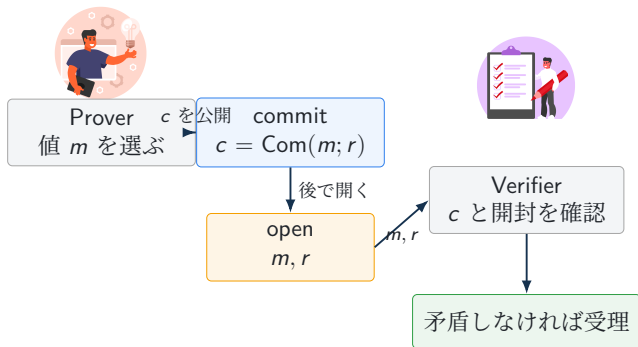
$$c \leftarrow \text{Com}(m; r), \quad \text{Verify}(c, m, r) \in \{0, 1\}$$

基本形では全体を開くが、Merkle tree や KZG では位置や点の値だけを補助情報つきで open する。

封筒の比喻: 封をした後で中身を変えられない。ただし開けるまでは中身を見せない。

Binding: commit 後に、別の値として open できない。

Hiding: open するまで、commitment から値が漏れない。



Commitment scheme の厳密な形

公開パラメータを使う構成では、次の三つのアルゴリズムとして書ける。

$$pp \leftarrow \text{Setup}(1^\lambda), \quad (c, d) \leftarrow \text{Commit}(pp, m), \quad b \leftarrow \text{Verify}(pp, c, m, d)$$

記号	意味
pp	公開パラメータ
c	commitment
d	decommitment / opening information。乱数、path、proof などの確認用補助情報
b	検証結果。受理なら 1、拒否なら 0

Binding

効率的な攻撃者が、同じ commitment を二つの異なる message として open することは困難である。このような組を見つける確率は negligible である。

$$\text{Verify}(pp, c, m, d) = 1,$$

$$\text{Verify}(pp, c, m', d') = 1, \quad m \neq m'$$

Hiding

c だけを見ても、どちらの message から作られたかを当てにくい。

$$\left| \Pr[b' = b] - \frac{1}{2} \right| \text{ が小さい}$$

Hash commitment の定義

Hash commitment は、message m を hash 値 c に固定する commitment である。

$$c = \text{Hash}(m)$$

prover は c を先に公開する。後で open するとき
は m を渡し、verifier は

$$\text{Hash}(m) = c$$

を確認する。

- commitment は hash 値 1 個で短い。
- binding は hash の第二原像困難性や衝突困難性に依存する。
- hiding は自動では得られない。

Hash commitment の安全性

Hash commitment では

$$c = \text{Hash}(m)$$

として値を固定する。

性質	意味
一方向性	$h = \text{Hash}(m)$ から元の m を見つけにくい。
第二原像困難性	与えられた m と同じ hash になる別の $m' \neq m$ を見つけにくい。
衝突困難性	異なる m, m' で $\text{Hash}(m) = \text{Hash}(m')$ となる組を見つけにくい。

binding には主に第二原像困難性や衝突困難性が効く。

Hash commitment で値を隠すには

候補が少ない message では、検証者が候補を全部 hash して当てられる。例えば $m \in \{0, 1\}$ なら探索が簡単。

乱数を混ぜる

秘匿性が必要な場合は、十分長い乱数 r を混ぜる。

$$c = \text{Hash}(\text{encode}(\text{domain}, r, m))$$

- domain: 用途を区別する文字列として入れる。
- encode: 長さや区切りが曖昧にならない一意な符号化を使う。

Merkle tree

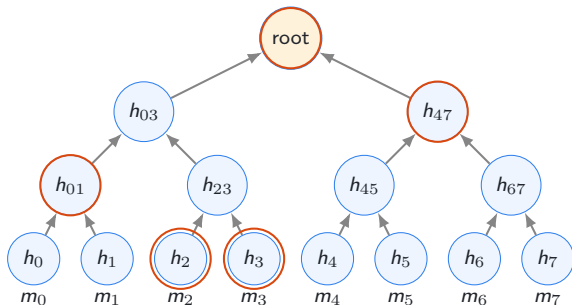
大きな配列 $(m_0, m_1, \dots, m_{n-1})$ 全体を固定したいとき、配列全体を毎回 open すると検証の負担が大きい。Merkle tree は、葉から親へ hash を重ねて、配列全体を 1 つの root にまとめる。

root を commitment として公開する。

$$h_i = \text{Hash}(m_i), \quad \text{root} = \text{Hash}(\dots)$$

1 要素だけを open するときは、root まで再計算するために必要な隣接 hash だけを渡す。

Merkle tree では、配列の一部だけを $O(\log n)$ の proof で open できる。



Merkle proofで検証者が読むもの

8 個の leaf からなる木で、 m_2 の inclusion proof を示す。

$$h_i = \text{Hash}(m_i)$$

$$h_{2,3} = \text{Hash}(h_2 \| h_3)$$

$$h_{0,1,2,3} = \text{Hash}(h_{0,1} \| h_{2,3})$$

$$\text{root} = \text{Hash}(h_{0,1,2,3} \| h_{4,5,6,7})$$

root と index 2 が公開されているとき、open する値と path として必要な値は次である。

$$m_2, \quad (h_3, \text{right}), \quad (h_{0,1}, \text{left}), \quad (h_{4,5,6,7}, \text{right})$$

実装上の注意

左右の順序、leaf のエンコード、domain separation を曖昧にしない。

Pedersen commitment

楕円曲線上の基点 G, H を使う。

$$\text{Com}(m; r) = mG + rH$$

Hiding

乱数 r によって、同じ m でも異なる点になる。

Binding

G と H の離散対数関係が知られていないことが重要になる。

加法準同型性

$$\text{Com}(a; r_a) + \text{Com}(b; r_b) = \text{Com}(a + b; r_a + r_b)$$

Pedersen で注意すること

もし同じ commitment を二通りに open できるなら、

$$mG + rH = m'G + r'H$$

なので、

$$(m - m')G = (r' - r)H$$

$H = \alpha G$ の α を知っていると、別の開き方を作れる可能性がある。

もう一つの注意

値は群の位数を法として扱われる。整数としての範囲を保証したい場合は range proof や追加制約が必要になる。

SRS

SRS は Structured Reference String の略で、証明者と検証者が共通に使う公開パラメータである。KZG では、setup で秘密の τ を選び、 τ から作った群要素だけを公開する。

$$\text{SRS} = (P, \tau P, \tau^2 P, \dots, \tau^d P; Q, \tau Q)$$

commitment と proof 生成には $\tau^i P$ のような G_1 側の群要素を使う。prover も verifier もこれを使う。ただし、群要素から τ そのものを取り出せないことを使っている。

pairing 検証では、検証用に $Q, \tau Q$ も公開される。 τ そのものは知られてはいけない。 τ が漏れると、主に binding が壊れ、不正な opening proof を作れる可能性がある。

trusted setup は、 τ を安全に生成し、最後に破棄したと信じる必要がある設定である。この秘密は toxic waste と呼ばれることがある。

KZG commitment

KZG は、多項式 $f(X)$ を短い commitment C に固定し、後で 1 点での値 $f(z) = y$ を短い proof で示す多項式 commitment である。

SRS として次の群要素が公開されているとする。

$$P, \tau P, \tau^2 P, \dots, \tau^d P; \quad Q, \tau Q$$

多項式を次のように書く。

$$f(X) = a_0 + a_1 X + \dots + a_d X^d$$

この多項式の commitment は

$$C = a_0 P + a_1 (\tau P) + \dots + a_d (\tau^d P) = f(\tau) P$$

証明者も検証者も τ 自体は知らない。公開された群要素を線形結合することで、 $f(\tau)P$ に相当する点を作る。

KZG opening

$f(z) = y$ を示したい。因数定理より、 $f(X) - y$ は $X - z$ で割り切れる。

$$q(X) = \frac{f(X) - y}{X - z}$$

KZG proof は

$$\pi = q(\tau)P$$

検証式:

$$e(C - yP, Q) = e(\pi, \tau Q - zQ)$$

正しく作った proof なら、completeness は次の式変形で確認できる。

$$\begin{aligned} e(C - yP, Q) &= e((f(\tau) - y)P, Q) \\ &= e(q(\tau)(\tau - z)P, Q) \\ &= e(q(\tau)P, (\tau - z)Q) \\ &= e(\pi, \tau Q - zQ). \end{aligned}$$

KZG の性質

KZG は、多項式全体を verifier へ送らずに、1 点または複数点での値を短く示すために使う。

基本版

- $C = f(\tau)P$ 、 $\pi = q(\tau)P$ はどちらも群要素 1 個である。
- proof 要素 π は定数サイズである。ただし開く値や検証計算は評価点数に依存する。

hiding 版

独立な基点 H と乱数多項式 $r(X)$ を使う。SRS には $H, \tau H, \dots$ 側の powers も必要になる。

$$C = f(\tau)P + r(\tau)H$$

$f(z) = y$, $r(z) = \rho$ を開くときは

$$\pi = q_f(\tau)P + q_r(\tau)H$$

$$q_f(X) = \frac{f(X) - y}{X - z}, \quad q_r(X) = \frac{r(X) - \rho}{X - z}$$

$$e(C - yP - \rho H, Q) = e(\pi, \tau Q - zQ)$$

batch 版

複数の点 $z_1, \dots, z_m$ で $f(z_i) = y_i$ をまとめて示す。補間多項式 $Y(X)$ と積多項式 $D(X)$ を置く。

$$Y(z_i) = y_i, \quad D(X) = \prod_{i=1}^m (X - z_i)$$

$$q(X) = \frac{f(X) - Y(X)}{D(X)}, \quad \pi = q(\tau)P$$

検証は 1 つの pairing 等式で行う。

$$e(C - Y(\tau)P, Q) = e(\pi, D(\tau)Q)$$

この式をそのまま使うには、 $D(\tau)Q$ を作るための G_2 側 powers も必要になる。

setup の秘密 τ が漏れると、不正な opening proof を作れる可能性がある。

Commitment scheme の比較

Scheme	対象	主な仮定	証明サイズ	強み	注意点
Hash	1 個の message	第二原像困難性、衝突困難性	open 対象に依存	単純	小さい message は乱数が必要になる
Merkle	配列・木	第二原像困難性、衝突困難性	$O(\log n)$	部分 open が得意	順序、salt、domain separation
Pedersen	値・ベクトル	離散対数問題、基点間の未知関係	開く値に依存	hiding、加法準同型	乱数再利用、範囲制約
KZG	多項式	ペアリング、SDH 系仮定、SRS	π は $O(1)$	短い proof、高速検証	opening data と検証計算は点数に依存

最低限覚えること

Merkle はデータ構造、Pedersen は準同型 commitment、KZG は多項式 commitment として覚える。

Arithmetization の目的

arithmetization は、

計算が正しく実行された

という主張を、有限体上の制約として表す工程である。
zk アプリケーション開発では、「自分が書いたコードがどんな制約になるか」を意識することが重要になる。



Prover が扱う計算を
制約へ落とす

制約は等価性の基準

制約は、候補の値を代入したときに成り立つべき条件である。計算結果をもう一度全部実行する代わりに、値どうしの整合性を等式で確認する。

計算として読む

$$a + b = c$$

c は $a + b$ の結果である。

制約として読む

$$a + b - c = 0$$

与えられた a, b, c が整合しているかを確認する。

制約は「0になる式」として書く

verifier が確認しやすいように、成り立つべき関係を「左辺が 0 になる式」へそろえる。

$$a + b = c \iff a + b - c = 0,$$

$$a \cdot b = d \iff ab - d = 0,$$

$$b \in \{0, 1\} \iff b(b - 1) = 0.$$

AND を表すなら、たとえば

$$c = a \wedge b \implies ab - c = 0$$

と書く。さらに a, b, c が boolean であることも別に制約として入れる。

witness と中間値を制約へ分解する

次の計算を証明したい。

$$x^3 + x + 5 = y$$

verifier にも見える値は y であり、prover は秘密入力 x と中間値 t_1, t_2 を witness として持つ。

public input: $y = 15$

witness variables: x, t_1, t_2

計算ステップへ分解する

$$t_1 \leftarrow x \cdot x$$

$$t_2 \leftarrow t_1 \cdot x$$

$$y \leftarrow t_2 + x + 5$$

たとえば $x = 2$ なら、

$$t_1 = 4, \quad t_2 = 8, \quad y = 15$$

制約として読む

$$t_1 - x^2 = 0$$

$$t_2 - t_1 x = 0$$

$$y - t_2 - x - 5 = 0$$

これらがすべて 0 なら、提出された x, t_1, t_2 は公開値 y と整合している。

線形結合とは何か

R1CS では「線形結合」という言葉が何度も出る。これは変数を係数付きで足した式である。

$$3z_1 + 5z_2 - z_4 + 7$$

定数項を扱いやすくするため、変数ベクトルの先頭に 1 を入れることが多い。

$$z = (1, y, x, t_1, t_2)$$

例

$t_2 + x + 5 - y$ は、 $z = (1, y, x, t_1, t_2)$ に対する線形結合として書ける。

$$5 \cdot 1 - 1 \cdot y + 1 \cdot x + 0 \cdot t_1 + 1 \cdot t_2$$

R1CS: 掛け算 1 個の制約へ分解する

R1CS の 1 本の制約は、

$$\text{線形結合} \times \text{線形結合} = \text{線形結合}$$

の形で書く。
変数ベクトル

$$z = (1, \text{public inputs, witness variables})$$

を用意し、

$$\langle A_i, z \rangle \cdot \langle B_i, z \rangle = \langle C_i, z \rangle$$

を全ての i で満たすかを見る。

感覚

R1CS は、算術回路を「掛け算 1 個を含む等式」の集まりへ分解する見方である。

R1CS で小例を行列として書く

$x^3 + x + 5 = y$ では $z = (1, y, x, t_1, t_2)^\top$ とおく。3本の制約を、3行の行列でまとめて書ける。

$$A = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 5 & -1 & 1 & 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad C = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

R1CS は次の形で確認する。

$$(Az) \circ (Bz) = Cz$$

実際に掛けると、

$$\begin{pmatrix} x \\ t_1 \\ t_2 + x + 5 - y \end{pmatrix} \circ \begin{pmatrix} x \\ x \\ 1 \end{pmatrix} = \begin{pmatrix} t_1 \\ t_2 \\ 0 \end{pmatrix}.$$

これは $x \cdot x = t_1$ 、 $t_1 \cdot x = t_2$ 、 $t_2 + x + 5 - y = 0$ を表す。

同じ計算を違う形に並べる

ここから出る方式は、別々の数学をしているように見えるが、目的は同じ。

方式	何として並べるか	見方
R1CS	掛け算 1 個の制約リスト	回路を細かい等式へ分解する
AIR	時間方向の trace table	実行ログの各行が正しいか見る
Plonkish	wire 列、gate、selector の表	表計算のように行ごとのルールを見る

共通点

どの方式でも、計算の正しさを有限体上の制約として扱う。

AIR: 実行トレースと遷移制約

AIR は、計算を「横に変数、縦に時間」を持つ表として見る。

step	a	b
0	0	1
1	1	1
2	1	2
3	2	3

Fibonacci の遷移制約:

$$a_{i+1} = b_i, \quad b_{i+1} = a_i + b_i$$

R1CS は回路を見る。AIR は実行ログが正しいかを見る。

Plonkish: 表として制約を書く

Plonkish arithmetization では、計算を表として配置する。

row	a	b	c	q_{mul}	q_{add}
0	a_0	b_0	c_0	1	0
1	a_1	b_1	c_1	0	1
2	a_2	b_2	c_2	1	0

- row は計算ステップを置く行である。
- a, b, c は wire 値を置く advice column であり、prover が witness や中間値から埋める。
- $q_{\text{mul}}, q_{\text{add}}$ は fixed column に置く selector であり、その行でどの gate を有効にするかを選ぶ。gate は各行で満たすべき式である。
- 公開入力 instance column に置く。別々のセルが同じ値であることは copy constraint で保証する。

Plonkish の gate を式で見る

典型的な gate:

$$q_M ab + q_L a + q_R b + q_O c + q_C = 0$$

gate	q_M	q_L	q_R	q_O	q_C	得られる制約
mul	1	0	0	-1	0	$ab - c = 0$
add	0	1	1	-1	0	$a + b - c = 0$
const	0	1	0	0	$-k$	$a - k = 0$

ここで $A, B, C, Q_M, Q_L, Q_R, Q_O, Q_C$ は、各列を補間して多項式として見たものである。列全体では

$$G(X) = Q_M AB + Q_L A + Q_R B + Q_O C + Q_C$$

が評価領域 H 上で 0 になることを要求する。

列を多項式として扱う

行数を n 、評価領域を

$$H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\}$$

とする。

列 a は、各行の値 a_i を満たす多項式 $A(X)$ として表す。

$$A(\omega^i) = a_i$$

同様に $B, C, Q_M, Q_L, Q_R, Q_O, Q_C$ も多項式になる。各行では、gate の式

$$G(\omega^i) = 0$$

を確認したい、という形で読む。

Plonkish で小例を配置する

$x^3 + x + 5 = y$ を 3 本の advice 列 a, b, c に置く。

row	a	b	c	gate
0	x	x	t_1	$a \cdot b = c$
1	t_1	x	t_2	$a \cdot b = c$
2	x	5	t_3	$a + b = c$
3	t_2	t_3	y	$a + b = c$

copy constraint

row 0 の c と row 1 の a は同じ t_1 。別々のセルが同じ値であることを、copy constraint で保証する。また row 2 の $b = 5$ は、fixed column や constant gate で固定する。

小例で必要な copy constraint

$x^3 + x + 5 = y$ の配置では、同じ値として結ぶセルが複数ある。

t_1 : row 0 の c と row 1 の a

x : row 0 の a, b , row 1 の b , row 2 の a

t_2 : row 1 の c と row 3 の a

t_3 : row 2 の c と row 3 の b

y : row 3 の c と公開入力 y

式としては

$$c_0 = a_1, \quad a_0 = b_0 = b_1 = a_2, \quad c_1 = a_3, \quad c_2 = b_3, \quad b_2 = 5, \quad c_3 = y_{\text{public}}$$

なぜ copy constraint が必要か

表のセルは場所が違えば、基本的には別の変数。同じ名前に見える値でも、同じであることを制約で言わなければならない。

小例

row 0 で計算した t_1 を、row 1 で使いたい。

$$\text{row 0 の } c = t_1, \quad \text{row 1 の } a = t_1$$

この 2 つのセルが同じ値だと保証するのが copy constraint。

注意

表に同じ記号を書いただけでは、数学的には同じ値だと証明されない。equality も制約として入れる必要がある。

Lookup とは何か

lookup は、証明中で使った値が、あらかじめ固定した表に含まれることを示す制約である。複雑な計算を毎回 gate で展開する代わりに、「この行は許可表のどれかと一致する」と確認する。

証明中で使った値

$$v_0, v_1, \dots, v_{r-1}$$



固定された許可表

$$T = \{0, 1, \dots, 255\}$$

$$v_i \in T \quad (i = 0, \dots, r-1)$$

を短い lookup argument で証明する。

- 表 T は fixed column として回路側に置く。
- 問い合わせる値 v_i は advice column に置く。
- 実装では並べ替えや多重集合の一致を使うが、意味は table membership である。

値の組も同じ考え方で扱える。例えば $(\text{opcode}, a, b, c) \in T_{\text{op}}$ と書けば、1 行の命令が許可された命令表に載っていることを示す。

Lookup の応用例

lookup は、「小さな表で表せる関係」を回路の中で何度も使うときに効く。

用途	表に置くもの	何を簡潔に書けるか
range check	$T_{\text{byte}} = \{0, 1, \dots, 255\}$	$v \in T_{\text{byte}}$ と書くことで、byte 値であることを示す。bit ごとの制約を毎回並べない。
ハッシュ関数の部品	$T_{\text{sbox}} = \{(x, S(x))\}$	S-box や小さな非線形変換を、入力と出力の対応表として確認する。
zkVM の命令表	$T_{\text{op}} = \{(\text{op}, a, b, c)\}$	$(\text{op}, a, b, c) \in T_{\text{op}}$ として、opcode ごとの意味を表にまとめる。

lookup だけで計算全体の正しさが決まるわけではない。gate は行ごとの等式を確認し、copy constraint はセル同士の同一性を確認し、lookup は値が許可表に入っていることを確認する。

CCS: 制約形式を一般化する

CCS は Customizable Constraint Systems の略。

R1CS は

$$(Az) \circ (Bz) - Cz = 0$$

と書ける。

CCS ではより一般に、

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0$$

の形で制約を表す。ここで S_i は項に使う添字を指定する集合であり、厳密には同じ添字を複数回使える多重集合として扱う。

何が嬉しいか

R1CS、Plonkish 風の gate、高 degree gate を、線形結合の積の和として同じ記法で扱える。

CCS のデータ

有限体を $\mathbb{F}$ とする。CCS では次のデータを固定する。

- 行数 m と変数数 n 。
- 行列 $M_1, \dots, M_t \in \mathbb{F}^{m \times n}$ 。
- 項を指定する添字集合 $S_1, \dots, S_q$ 。厳密には多重集合として扱う。
- 係数 $c_1, \dots, c_q \in \mathbb{F}$ 。

公開入力 x 、witness w 、定数 1 を含めて

$$z = (w, x, 1) \in \mathbb{F}^n$$

と置く。順番は方式や文献ごとの約束であり、大事なものは必要な値をすべて z に含めることである。

CCS の式を成分ごとに読む

CCS の制約は

$$\sum_{i=1}^q c_i \cdot \bigcirc_{j \in S_i} (M_j z) = 0 \in \mathbb{F}^m$$

である。

Hadamard 積は成分ごとの積:

$$(u \circ v)_r = u_r v_r$$

成分ごとに書くと、各行 r について

$$\sum_{i=1}^q c_i \prod_{j \in S_i} (M_j z)_r = 0$$

を要求している。

R1CS は CCS の特別な場合

R1CS は

$$(Az) \circ (Bz) - Cz = 0$$

と書ける。

CCS で

$$M_1 = A, \quad M_2 = B, \quad M_3 = C$$

$$S_1 = \{1, 2\}, \quad S_2 = \{3\}, \quad c_1 = 1, \quad c_2 = -1$$

と置くと、

$$(M_1 z) \circ (M_2 z) - (M_3 z) = 0$$

となり、R1CS を含む。

Plonkish gate も CCS で見る

Plonkish の gate

$$q_{Mab} + q_L a + q_R b + q_O c + q_C = 0$$

も、行ごとに見れば線形結合の積の和。

$$(M_1 z) \circ (M_2 z) + (M_3 z) + (M_4 z) + (M_5 z) + (M_6 z) = 0$$

高 degree gate も同じ見方で扱える。例えば

$$a_i b_i c_i - d_i = 0$$

は

$$(M_1 z) \circ (M_2 z) \circ (M_3 z) - (M_4 z) = 0$$

と書ける。

CCS が folding で役立つ理由

folding scheme では、二つの証明対象を一つの証明対象へまとめる。そのとき、制約が共通の形で書かれていると扱いやすい。

$$\sum_i c_i \cdot \prod_{j \in S_i} \text{linear form}_{i,j}(z) = 0$$

分解できる操作

- 線形結合を作る。
- 同じ行どうしで掛ける。
- 係数を掛けて足す。

R1CS、Plonkish 風の gate、高 degree gate を共通インターフェースへ持ち込みやすい。

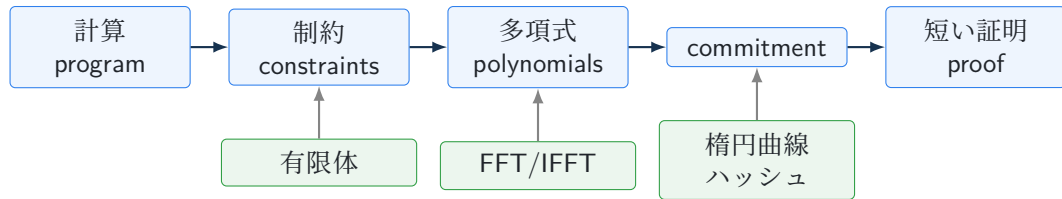
Arithmetization の比較

方式	主な表現	得意な計算	よく出る文脈	理解
R1CS	乗算制約の集合	算術回路	Groth16、Circom	回路を制約に分解する
AIR	実行トレースと遷移	VM、繰り返し計算	STARK	計算履歴が正しいか見る
Plonkish	gate と copy constraint	汎用回路、lookup	Plonk 系	表計算のように制約を書く
CCS	一般化された制約	folding-friendly な表現	Nova 系	複数方式をまとめる抽象化

最低限覚えること

arithmetization は、計算を有限体上の制約へ翻訳する工程である。方式ごとに「何を自然に表せるか」が違う。

全体像をもう一度



一言で言うと

zkSNARK は、公開 statement に対して秘密 witness が存在することを、計算を制約と多項式へ翻訳し、commitment で短く固定して、短い proof として検証する仕組みである。

同じ計算を複数の見方で見ると

$$x^3 + x + 5 = y$$

- R1CS: $t_1 = x^2$, $t_2 = t_1 x$ のように掛け算制約へ分解する。
- AIR: 計算を step ごとの状態遷移として並べる。
- Plonkish: gate、wire、selector、copy constraint として表に配置する。
- CCS: 線形結合の積の和として、複数の制約形式を統一的に見る。

変わらない目的は、公開入力と秘密入力を分け、計算の正しさを有限体上の制約で示すことである。

Further Reading

- RareSkills ZK Book
- MoonMath Manual
- SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge
- KZG commitment paper
- Ethereum EIP-4844: blob transactions
- Ethereum consensus specs: polynomial commitments
- 入門チュートリアル: Circom, Noir, Halo2 book



記法、用語

記法

記号	意味
x	公開入力。verifier にも見える値。
w	witness。statement を成り立たせる補助情報。
π	proof。verifier へ渡す短い証拠。
$R(x, w) = 1$	関係 R が、公開入力 x と witness w を受理すること。 statement は通常 $\exists w : R(x, w) = 1$ 。
$\mathbb{F}_p$	素数 p 個の元からなる有限体。
$\mathbb{F}_p^*$	$\mathbb{F}_p$ から 0 を除いた集合。掛け算について群になる集合。
$\langle g \rangle$	元 g が生成する巡回群。
g^a, aP	乗法群と楕円曲線の加法群で、秘密スカラーを掛けた公開値。
$E/k, e$	体 k 上の楕円曲線と、 $e : G_1 \times G_2 \rightarrow G_T$ のようなペアリング。

用語

用語	意味
statement	検証者が確認したい公開された主張。
proof / argument	statement が正しいこと、または witness を知っていることを示す証拠。witness の秘匿は zero-knowledge 性が担う。
transcript hash	protocol 名、statement、commitment などの公開ログから作る hash。
commitment	値や多項式を短い値に固定し、必要な部分だけ open する仕組み。
binding / hiding	commit 後に別の値へ開けない性質と、commitment から値が漏れない性質。
arithmetization	計算を有限体上の制約へ翻訳する工程。
R1CS / AIR / Plonkish / CCS	計算を制約として表すための代表的な表現形式。
離散対数問題	公開値から秘密スカラーを求める問題。
計算	Diffie-Hellman 型の値を計算する問題。
Diffie-Hellman 問題	
判定	Diffie-Hellman 型の値かどうかを判定する問題。
Diffie-Hellman 問題	
SRS	KZG などを使う Structured Reference String。公開パラメータ。